Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра программного обеспечения и информационных технологий
Дисциплина «Метрология, стандартизация и сертификация»

ОТЧЕТ
к лабораторной работе №2
на тему:
«МЕТРИКИ СЛОЖНОСТИ ПОТОКА УПРАВЛЕНИЯ ПРОГРАММ»
БГУИР 6-05-0612-02

Выполнили студенты группы 453503
ХАЛАМОВ Николай Андреевич
ХАДОРИК Матвей Денисович

_____
(дата, подпись студента)

Проверил ассистент каф. ПОИТ
БОЛТАК Светлана Владимировна

_____
(дата, подпись преподавателя)

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Задание

1 Изучить метрики сложности потока управления программ.

2 Взять код программы в консольном режиме на языке Python.

3 Написать свою программу (на любом языке программирования), которая будет анализировать программу из пункта 2. На основе анализа программа должна рассчитать метрики Джилба согласно примеру в файле из пункта 1. Результат работы программы: вывести на экран 2 основные метрики Джилба и 1 расширенную в удобном для просмотра виде (например, в таблицу).

2 ВЫПОЛНЕНИЕ РАБОТЫ

Ниже приведён код для расчёта метрик Джилба

```
require 'tk'

class RubyGilbAnalyzer
  attr_reader :total_operators, :conditional_operators, :max_nesting_level

  def initialize
    @total_operators = 0
    @conditional_operators = 0
    @max_nesting_level = 0
    @current_nesting = 0
    @operator_counts = {}
  end

  def clean_code(code)
    in_multiline = false
    result = []

    code.each_line do |line|
      s = line.strip

      if s.start_with?("=begin")
        in_multiline = true
        result << ""
        next
      end
      if s.start_with?("=end")
```

```
  in_multiline = false
  result << ""
  next
end
if in_multiline
  result << ""
  next
end

cleaned = ""
in_sq = false
in_dq = false
i = 0
chars = line.chars

while i < chars.length
  ch = chars[i]

  if in_sq
    if ch == '\\' && i + 1 < chars.length
      cleaned << "__"
      i += 2
      next
    end
    if ch == "'"
      in_sq = false
    end
    cleaned << "_"
    i += 1
    next
  end

  if in_dq
    if ch == '\\' && i + 1 < chars.length
      cleaned << "__"
      i += 2
      next
    end
    if ch == '"'
      in_dq = false
    end
    cleaned << "_"
    i += 1
    next
  end

  if ch == "'"
    in_sq = true
    cleaned << "_"
```

```
          i += 1
          next
        end

        if ch == '"'
          in_dq = true
          cleaned << "_"
          i += 1
          next
        end

        if ch == '#'
          break
        end

        cleaned << ch
        i += 1
      end

      result << cleaned
    end

    result.join("\n")
  end

  def add_operator(name, count = 1)
    @operator_counts[name] = (@operator_counts[name] || 0) + count
    @total_operators += count
  end

  def add_conditional(name, count = 1)
    @conditional_operators += count
  end

  def analyze(code)
    @total_operators = 0
    @conditional_operators = 0
    @max_nesting_level = 0
    @current_nesting = 0
    @operator_counts = {}

    cleaned = clean_code(code)
    lines = cleaned.split("\n")

    block_stack = []
    case_when_counts = []
    # Для отслеживания вложенности case: сохраняем уровень на момент входа в
case
    case_nesting_base = []
```

```
    lines.each do |line|
      stripped = line.strip
      next if stripped.empty?

      # ── end ──
      if stripped =~ /^end\b/
        if block_stack.length > 0
          block_type = block_stack.pop

          if block_type == "case"
            if case_when_counts.length > 0
              n_whens = case_when_counts.pop
              base_nesting = case_nesting_base.pop || @current_nesting

              if n_whens > 0
                equiv_ifs = n_whens - 1
                add_conditional("case/when (экв. #{equiv_ifs} if)", equiv_ifs)
                add_operator("case/when", 1)
                # case вносит 1 уровень за себя + (n-2) за внутреннюю
вложенность
                # итого эквивалентная глубина = n-1 уровней поверх base
                case_depth = [n_whens - 1, 0].max
                effective = base_nesting + case_depth
                @max_nesting_level = [@max_nesting_level, effective].max
              end
            end
            # Восстанавливаем nesting на уровень до case
            # (current_nesting не менялся при входе в case)
          elsif ["if", "unless", "while", "until", "for"].include?(block_type)
            @current_nesting -= 1
          end
          # def, class, module, begin, do — просто закрываем
        end
        next
      end

      # ── case ──
      if stripped =~ /^case\b/ || stripped =~ /=\s*case\b/
        block_stack.push("case")
        case_when_counts.push(0)
        case_nesting_base.push(@current_nesting)
        next
      end

      # ── when ──
      if stripped =~ /^when\b/
        if case_when_counts.length > 0
          case_when_counts[-1] += 1
```

```
  end
  next
end

# ── else, then, do (сами по себе) ──
if ["else", "then", "do"].include?(stripped)
  next
end

# ── if, unless, while, until, for в начале строки ──
if stripped =~ /^(if|unless|while|until|for)\b/
  kw = $1
  block_stack.push(kw)
  @current_nesting += 1
  @max_nesting_level = [@max_nesting_level, @current_nesting].max

  if kw == "while"
    add_operator("while", 1)
    add_conditional("while", 1)
  elsif kw == "until"
    add_operator("until", 1)
    add_conditional("until", 1)
  elsif kw == "for"
    add_operator("for", 1)
    add_conditional("for", 1)
  elsif kw == "if"
    add_operator("if", 1)
    add_conditional("if", 1)
  elsif kw == "unless"
    add_operator("unless", 1)
    add_conditional("unless", 1)
  end
  next
end

# ── elsif ──
if stripped =~ /^elsif\b/
  add_operator("elsif", 1)
  add_conditional("elsif", 1)
  next
end

# ── def ──
if stripped =~ /^def\b/
  block_stack.push("def")
  add_operator("def", 1)
  next
end
```

```
# ── class, module ──
if stripped =~ /^(class|module)\b/
  block_stack.push($1)
  add_operator($1, 1)
  next
end

# ── begin ──
if stripped =~ /^begin\b/
  block_stack.push("begin")
  add_operator("begin", 1)
  next
end

# ── rescue, ensure ──
if stripped =~ /^(rescue|ensure)\b/
  add_operator($1, 1)
  next
end

# ── Проверяем do-блок в конце строки ──
has_do_block = !!(stripped =~ /\bdo\s*(\|[^|]*\|)?\s*$/)
if has_do_block
  block_stack.push("do")
end

# ── Теперь считаем оператор строки ──

# return
if stripped =~ /^return\b/
  add_operator("return", 1)
  check_postfix(stripped)
  next
end

# break
if stripped =~ /^break\b/
  add_operator("break", 1)
  next
end

# next
if stripped =~ /^next\b/
  add_operator("next", 1)
  check_postfix(stripped)
  next
end

# redo, retry
```

```
      if stripped =~ /^(redo|retry)\b/
        add_operator($1, 1)
        next
      end

      # raise
      if stripped =~ /^raise\b/
        add_operator("raise", 1)
        check_postfix(stripped)
        next
      end

      # yield
      if stripped =~ /^yield\b/
        add_operator("yield", 1)
        check_postfix(stripped)
        next
      end

      # require, include, extend, attr_*, etc.
      if stripped =~
/^(require|require_relative|include|extend|attr_reader|attr_writer|attr_acces
sor|alias|undef|load|autoload)\b/
        add_operator($1, 1)
        next
      end

      # puts, print, p, gets и другие встроенные вызовы
      if stripped =~
/^(puts|print|p|gets|warn|abort|exit|sleep|rand|srand|sprintf|format)\b/
        add_operator($1, 1)
        check_postfix(stripped)
        next
      end

      # Присваивание (включая +=, ||= и т.д.)
      if stripped =~ /(?<![=!<>])[\w\]\)\.]+\s*(\+|-
|\*|\/|%|\*\*|\|\|\||&&|<<|>>|&|\||\^)?=(?!=)/
        add_operator("присваивание (=)", 1)
        check_postfix(stripped)
        next
      end

      # Вызов метода с блоком: something.each do, 3.times do
      if has_do_block
        add_operator("вызов метода с блоком", 1)
        check_postfix(stripped)
        next
      end
```

```
      # Вызов метода: obj.method(...) или method(args)
      if stripped =~ /^[A-Za-z_]\w*\s*[\.\(]/ || stripped =~ /^[A-Za-z_]\w*$/
        word = stripped.match(/^([A-Za-z_]\w*)/)[1]
        keywords = %w[end else elsif when then do in and or not true false
nil self
                      if unless while until for case def class module begin
                      rescue ensure return break next redo retry yield
                      raise require include puts print p gets]
        unless keywords.include?(word)
          add_operator("вызов метода", 1)
          check_postfix(stripped)
          next
        end
      end

      # Вызов метода на объекте: @var.method, $var.method, var.method
      if stripped =~ /^[@$]?\w+\./
        add_operator("вызов метода", 1)
        check_postfix(stripped)
        next
      end

      # Всё остальное непустое
      if stripped.length > 0 && stripped !~ /^(end|else|then|do|when)\b/
        add_operator("прочее", 1)
        check_postfix(stripped)
      end
    end

    # Обработка незакрытых case
    while case_when_counts.length > 0
      n_whens = case_when_counts.pop
      base_nesting = case_nesting_base.pop || @current_nesting
      if n_whens > 0
        equiv_ifs = n_whens - 1
        add_conditional("case/when", equiv_ifs)
        add_operator("case/when", 1)
        case_depth = [n_whens - 1, 0].max
        effective = base_nesting + case_depth
        @max_nesting_level = [@max_nesting_level, effective].max
      end
    end

    cl_val = @conditional_operators
    total_val = @total_operators
    relative_val = total_val > 0 ? (cl_val.to_f / total_val).round(4) : 0.0
    cli_val = @max_nesting_level
```

```
    {
      "CL" => cl_val,
      "cl" => relative_val,
      "CLI" => cli_val,
      "total" => total_val,
      "details" => @operator_counts
    }
  end

  private

  def check_postfix(stripped)
    # Ищем постфиксные if/unless/while/until (не в начале строки)
    stripped.scan(/\b(if|unless|while|until)\b/) do |match|
      m = Regexp.last_match
      pos = m.begin(0)
      if pos > 0
        before = stripped[0...pos].strip
        if before.length > 0 && !before.match?(/^(els)?$/)
          add_operator("постфиксный #{m[1]}", 1)
          add_conditional("постфиксный #{m[1]}", 1)
        end
      end
    end
  end
end


# ============ GUI ============

root = TkRoot.new do
  title "Анализатор метрик Джилба (Ruby)"
  minsize(850, 650)
end

TkLabel.new(root) do
  text "Анализатор метрик Джилба для Ruby"
  font TkFont.new("size" => 14, "weight" => "bold")
  pack("pady" => 10)
end

TkLabel.new(root) do
  text "Исходный код Ruby:"
  font TkFont.new("size" => 10)
  pack("anchor" => "w", "padx" => 10)
end

code_frame = TkFrame.new(root).pack("padx" => 10, "pady" => 5, "fill" =>
"both", "expand" => true)
```

```
code_scroll = TkScrollbar.new(code_frame)
code_text = TkText.new(code_frame) do
  width 100
  height 18
  font TkFont.new("family" => "Courier", "size" => 10)
  wrap "word"
  yscrollcommand proc { |*args| code_scroll.set(*args) }
end
code_scroll.command proc { |*args| code_text.yview(*args) }
code_scroll.pack("side" => "right", "fill" => "y")
code_text.pack("side" => "left", "fill" => "both", "expand" => true)

btn_frame = TkFrame.new(root).pack("pady" => 5)

TkButton.new(btn_frame) do
  text "Загрузить файл (.rb)"
  font TkFont.new("size" => 10)
  command proc {
    file = Tk.getOpenFile("filetypes" => [["Ruby файлы", "*.rb"], ["Все
файлы", "*"]])
    if file && file.length > 0
      begin
        content = File.read(file, encoding: "utf-8")
        code_text.delete("1.0", "end")
        code_text.insert("end", content)
      rescue => e
        Tk.messageBox("type" => "ok", "icon" => "error",
                      "title" => "Ошибка",
                      "message" => "Не удалось прочитать файл:\n#{e.message}")
      end
    end
  }
  pack("side" => "left", "padx" => 5)
end

result_text = nil

TkButton.new(btn_frame) do
  text "Анализировать"
  font TkFont.new("size" => 11, "weight" => "bold")
  background "#4CAF50"
  foreground "white"
  command proc {
    code = code_text.get("1.0", "end").strip
    if code.empty?
      Tk.messageBox("type" => "ok", "icon" => "warning",
                    "title" => "Ошибка",
                    "message" => "Пожалуйста, введите Ruby-код для анализа")
      next
```

```
    end

    begin
      analyzer = RubyGilbAnalyzer.new
      metrics = analyzer.analyze(code)

      sep = "=" * 62
      output = "#{sep}\n"
      output += "  РЕЗУЛЬТАТЫ АНАЛИЗА МЕТРИК ДЖИЛБА (Ruby)\n"
      output += "#{sep}\n\n"

      output += "+------------------------------------------------+----------
-+\n"
      output += "| Метрика                                        | Значение
|\n"
      output += "+------------------------------------------------+----------
-+\n"
      output += "| Абсолютная сложность (CL)                      |
#{metrics['CL'].to_s.center(9)} |\n"
      output += "| Относительная сложность (cl)                   |
#{metrics['cl'].to_s.center(9)} |\n"
      output += "| Макс. уровень вложенности (CLI)                |
#{metrics['CLI'].to_s.center(9)} |\n"
      output += "| Всего операторов                               |
#{metrics['total'].to_s.center(9)} |\n"
      output += "+------------------------------------------------+----------
-+\n"

      output += "\n  Таблица операторов:\n"
      output += "+------------------------------------------------+----------
-+\n"
      output += "| Оператор                                       | Кол-во
|\n"
      output += "+------------------------------------------------+----------
-+\n"

      details = metrics['details']
      details.sort_by { |k, v| -v }.each do |name, count|
        output += "| %-46s | %9s |\n" % [name, count]
      end

      output += "+------------------------------------------------+----------
-+\n"
      output += "| %-46s | %9s |\n" % ["ИТОГО", metrics['total']]
      output += "+------------------------------------------------+----------
-+\n"

      output += "\nПримечания:\n"
      output += "  CL  - количество условных операторов\n"
```

```
      output += "        (case/when c n when = n-1 условных операторов)\n"
      output += "  cl  - CL / общее число операторов\n"
      output += "  CLI - макс. уровень вложенности\n"
      output += "        (case/when c n when = вложенность n-1)\n"
      output += "        (while/if/unless/until/for = +1 уровень)\n"

      result_text.delete("1.0", "end")
      result_text.insert("end", output)
    rescue => e
      Tk.messageBox("type" => "ok", "icon" => "error",
                    "title" => "Ошибка анализа",
                    "message" => "Ошибка при анализе
кода:\n#{e.message}\n#{e.backtrace.first(5).join("\n")}")
    end
  }
  pack("side" => "left", "padx" => 5)
end

TkButton.new(btn_frame) do
  text "Очистить"
  font TkFont.new("size" => 10)
  command proc {
    code_text.delete("1.0", "end")
    result_text.delete("1.0", "end")
  }
  pack("side" => "left", "padx" => 5)
end

TkLabel.new(root) do
  text "Результаты анализа:"
  font TkFont.new("size" => 10)
  pack("anchor" => "w", "padx" => 10, "pady" => [5, 0])
end

result_frame = TkFrame.new(root).pack("padx" => 10, "pady" => [0, 10], "fill"
=> "x")
result_scroll = TkScrollbar.new(result_frame)
result_text = TkText.new(result_frame) do
  width 100
  height 18
  font TkFont.new("family" => "Courier", "size" => 10)
  wrap "word"
  yscrollcommand proc { |*args| result_scroll.set(*args) }
end
result_scroll.command proc { |*args| result_text.yview(*args) }
result_scroll.pack("side" => "right", "fill" => "y")
result_text.pack("side" => "left", "fill" => "both", "expand" => true)

Tk.mainloop
```

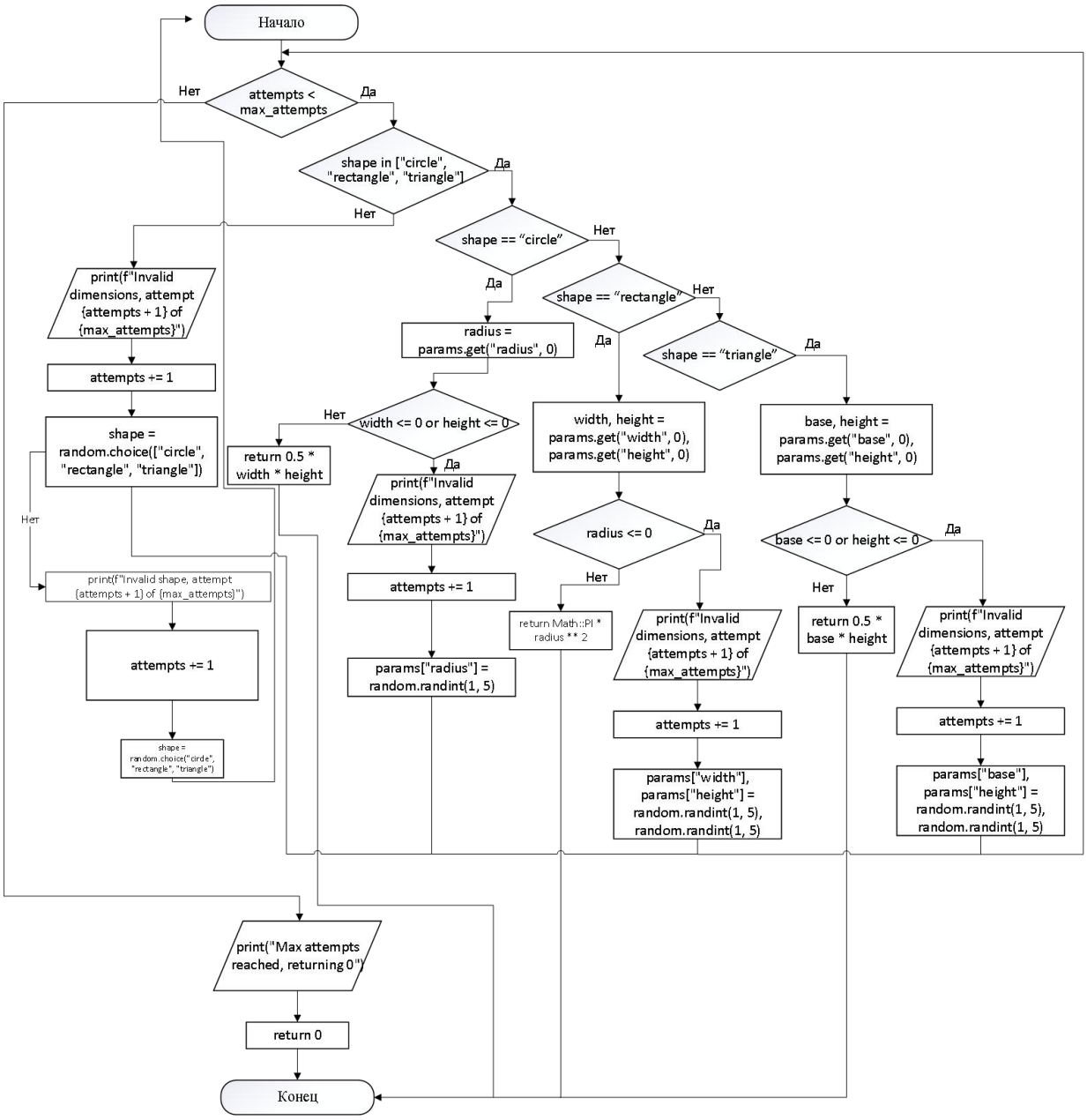


Рисунок 1 – Блок-схема части программы с максимальной вложенностью

Метрики, рассчитанные вручную:

j	Оператор	f_{Ij}
1	Присваивание (=)	25
2	Цикл while	1
3	puts	8
4	next	4
5	case/when	3
6	if	11
7	вызов метода	3
8	вызов метода с блоком	2
9	require	1
10	break	1
11	return	15
	Всего:	74

Количество условных операторов (CL): 18
Общее число операторов в программе: 74
Насыщенность программы условными операторами (cl): 18/74=0,2432
Максимальный уровень вложенности условного оператора (CLI): 5

```
======================================================================
  РЕЗУЛЬТАТЫ АНАЛИЗА МЕТРИК ДЖИЛБА (Ruby)
======================================================================

+-----------------------------------------------------+-----------+
| Метрика                                             | Значение  |
+-----------------------------------------------------+-----------+
| Абсолютная сложность (CL)                           |    18     |
| Относительная сложность (cl)                        |  0.2432   |
| Макс. уровень вложенности (CLI)                     |     5     |
| Всего операторов (N)                                |    74     |
+-----------------------------------------------------+-----------+
```

Рисунок 2 – Результат работы программы

ВЫВОД

В ходе лабораторной работы был проанализирован исходный код программы и изучены базовые и расширенные метрики Джилба сложности потока управления программ на примере языка программирования Python.